Report

Of

NetSage AI — An AI-Assisted Network Troubleshooting Assistant with Human-in-the-Loop Review

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Roll Number of Student: 25SCS1003004006

Name of Student: Tarun Pandey

Batch: 2026-27

CANDIDATE'S DECLARATION

I, **Tarun Pandey** do hereby declare that the internship report titled **"NetSage AI — An AI-Assisted Network Troubleshooting Assistant with Human-in-the-Loop Review"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature TARUN

Student Name ____Tarun Pandey_____

Roll No. ____25SCS1003004006_____

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to extend my sincere thanks to the faculty of the School of Computer Science and Engineering, IILM University, Greater Noida, for their continuous academic support and for providing the environment in which this project could be developed and evaluated. I am also grateful for the structured problem statement and technical direction that shaped the objectives of this work, which gave the project a clear industry-relevant focus.

I express my gratitude to my parents for their blessings.

Signature TARUN

Student Name ____Tarun Pandey____

Roll No. ____25SCS1003004006_____

INTERNSHIP COMPLETION CERTIFICATE


Cisco Networking Academy

IILM UNIVERSITY

This certificate is awarded to

Tarun Pandey

for successfully completing

Introduction to Modern AI

offered by IILM University
through the Cisco Networking Academy program.

Swati Vashisht
Instructor
IILM University

14 Jun 2026
Completion Date

Cert ID: 76b79dd3-aa9a-4f2b-9707-6777c83a7c47


Cisco Networking Academy

IILM UNIVERSITY

This certificate is awarded to

Tarun Pandey

for successfully completing

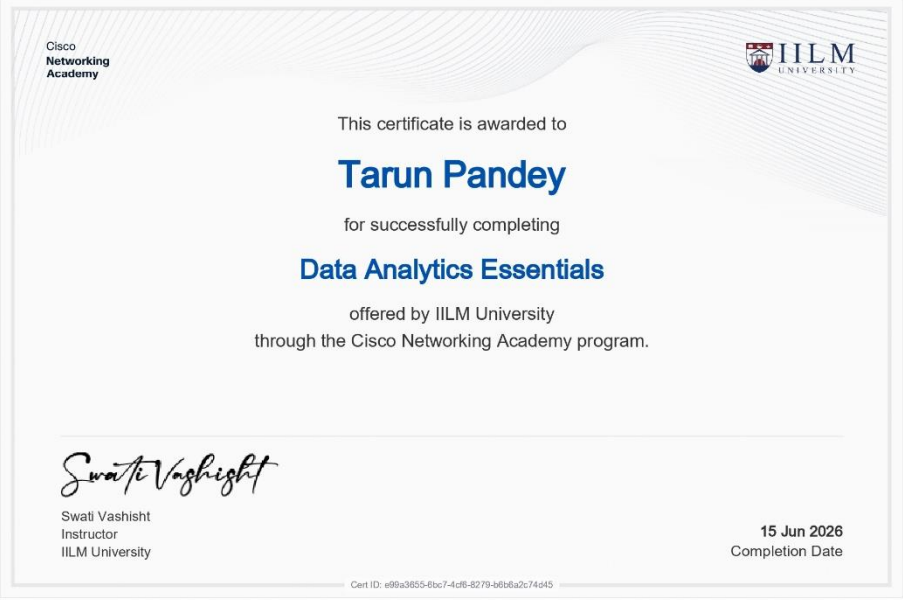
Apply AI: Analyze Customer Reviews

offered by IILM University
through the Cisco Networking Academy program.

Swati Vashisht
Instructor
IILM University

11 Jun 2026
Completion Date

Cert ID: 8bf38fcb-5809-442a-b30a-ab7d7030564b


Cisco Networking Academy

IILM UNIVERSITY

This certificate is awarded to

Tarun Pandey

for successfully completing

Data Analytics Essentials

offered by IILM University
through the Cisco Networking Academy program.

Swati Vashisht
Instructor
IILM University

15 Jun 2026
Completion Date

Cert ID: e99a3655-6bc7-4cf6-8279-b6b6a2c74d45

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

Computer networks form the backbone of every modern organisation, and the ability to diagnose network faults quickly is one of the most valuable practical skills a computer science graduate can possess. However, a persistent gap exists between knowing individual diagnostic commands and being able to reason from an observed symptom to an actual root cause. A junior network engineer may be perfectly capable of executing commands such as show ip interface brief or show vlan brief, yet still be unable to determine which of several possible faults is responsible for a reported failure.

This gap is most visible in a scenario that occurs repeatedly in laboratory environments and in production networks alike. A personal computer successfully obtains an IP address through DHCP but cannot reach a server located elsewhere in the network. The underlying cause could lie in any one of at least six distinct domains: an incorrect VLAN configuration, a missing or misconfigured route, a DHCP scope problem, a name-resolution failure, an access control list silently discarding the traffic, or an incomplete Network Address Translation configuration. Each of these possibilities requires a different diagnostic path, and choosing the wrong path costs time that may be critical during an outage.

NetSage AI was developed as a response to this problem. It is an AI-assisted troubleshooting helper for Cisco Packet Tracer laboratory scenarios that accepts a reported symptom, a short description of the topology, and the output of relevant show commands, and returns a structured diagnosis containing a likely root cause, the associated OSI layer, a confidence score, the specific evidence supporting the conclusion, the next diagnostic command to run, proposed remediation steps, and verification steps.

The defining characteristic of the system, and the principle around which its entire architecture is organised, is that the artificial intelligence component never has the final say. Every diagnosis the system produces is treated as a suggestion that must be explicitly reviewed by a human engineer, who accepts, edits, or rejects it before it is regarded as correct. This design pattern is known as human-in-the-loop artificial intelligence, and it directly addresses the well-documented tendency of large language models to produce fluent, confident, and entirely incorrect explanations. In a

networking context, acting on such an explanation could mean applying a configuration change that worsens an existing outage.

The project therefore has two intertwined goals. The first is technical: to build a working, demonstrable tool that genuinely assists with network fault diagnosis. The second is ethical and methodological: to demonstrate that an AI-assisted tool can be designed so that its limitations are visible, its mistakes are recorded rather than hidden, and human judgement remains authoritative at every stage.

4.2 Organization Profile

Cisco Systems, Inc. is an American multinational technology corporation headquartered in San Jose, California. Founded in 1984 by a group of computer scientists from Stanford University, the company has grown to become one of the largest and most influential networking hardware and software organisations in the world. Cisco designs, manufactures, and sells networking equipment including routers, switches, wireless access points, and security appliances, along with an extensive portfolio of enterprise software, collaboration platforms, and cybersecurity products.

Cisco holds a position of particular importance in the field of networking education. Through the Cisco Networking Academy programme, the organisation provides structured curricula, laboratory environments, and industry-recognised certification pathways to millions of students across the world. The Cisco Certified Network Associate credential is widely regarded as a foundational qualification for careers in network engineering, and the concepts examined by it, including VLANs, routing protocols, access control lists, and Network Address Translation, map directly onto the fault categories addressed by this project.

Cisco Packet Tracer, the simulation environment on which this project is based, is a network simulation tool developed by Cisco for educational purposes. It allows students to construct network topologies, configure devices using authentic Cisco IOS command syntax, and deliberately introduce faults in order to practise diagnostic reasoning. Because Packet Tracer reproduces genuine command output, a troubleshooting assistant developed against Packet Tracer scenarios operates on realistic evidence rather than artificial data.

The problem statement that defines this project originates from Cisco and is framed around building an AI troubleshooting helper with mandatory human review. The requirements specify a minimum dataset size, mandatory coverage of several distinct fault categories, a deterministic rule-based validation component, a structured prompt library, a human oversight workflow with explicit accept, edit, and reject decisions, and a documented record of cases in which the AI required correction. These requirements shaped the design decisions described throughout this report.

4.3 Problem Statement

The formal problem statement addressed by this project is to build an AI troubleshooting helper with human review: an AI-assisted troubleshooter for Packet Tracer laboratory problems that reads symptoms and show-command output, suggests likely causes and next steps, and always requires a human to review the diagnosis before the proposed fix is accepted.

The problem may be decomposed into four distinct sub-problems, each of which introduces its own technical difficulty.

The first is the symptom-to-cause mapping problem. A single observable symptom is consistent with many different underlying faults. The system must therefore reason across several fault domains simultaneously rather than pattern-matching to the most common explanation, and it must be capable of stating that the available evidence is insufficient to distinguish between competing hypotheses.

The second is the evidence-grounding problem. A language model asked to diagnose a network fault will readily produce a plausible-sounding explanation even when the supplied evidence does not support it, and may fabricate command output that was never provided. The system must therefore constrain the model to reason only from supplied evidence, cite the specific lines it relied upon, and explicitly identify what evidence is missing.

The third is the overconfidence problem. Language models express uncertainty poorly and frequently assign high confidence to incorrect conclusions. The system must produce a calibrated confidence score tied to the strength of the available evidence, and must lower that score when the evidence is thin, ambiguous, or internally contradictory.

The fourth is the oversight problem. If an incorrect diagnosis can be accepted automatically, the risk of the system causing harm scales with the frequency of its use. The system must therefore make human review structurally impossible to bypass, rather than merely recommending it as good practice.

A further constraint arises from the operating environment. The tool is intended for students working on ordinary laptops, and must therefore run without paid infrastructure, without containerisation, without a database server, and without requiring an API key in order to be demonstrated.

4.4 Project Objectives

The following objectives were defined at the start of the project and were used as the criteria against which the finished system was evaluated.

1. To construct a dataset of at least thirty realistic troubleshooting cases derived from Cisco Packet Tracer laboratory scenarios, distributed across the fault categories of VLAN, default gateway, DHCP, DNS, routing, access control lists, Network Address Translation, and wireless networking, with each case containing a symptom, a topology note, authentic show-command output, the expected fault, the associated OSI layer, a concept tag, a severity rating, the expected next diagnostic command, the expected fix, and a verification command.
2. To design and document a structured prompt library that constrains the AI to reason only from supplied evidence, to separate observation from inference, to return a fixed JSON schema containing root cause, confidence, evidence, next command, and fix steps, and to include worked examples demonstrating the complete diagnostic pipeline.
3. To implement a deterministic, rule-based configuration checker in Python, containing no AI component whatsoever, capable of detecting duplicate IP addresses, incorrect subnet masks, default gateway mismatches, administratively disabled interfaces, missing VLANs, and missing routes, and returning structured results with clear status indicators.
4. To build an AI diagnosis engine that is modular with respect to its underlying model, that reads its credentials exclusively from environment variables, and that degrades gracefully to a clearly labelled demonstration mode when no API key is configured, so that the project remains fully demonstrable without paid infrastructure.
5. To implement a human review workflow in which every AI diagnosis must be explicitly accepted, edited, or rejected by a named reviewer before it is considered final, with the requirement for review enforced in program code rather than merely requested in the prompt.
6. To maintain a Responsible AI log documenting at least five cases in which the initial AI diagnosis was incorrect, incomplete, or overconfident, recording for

each the actual fault, the correction applied, the reason the AI erred, and the lesson learned.

7. To develop a dashboard that reports the total number of cases, the distribution of cases by issue type and severity, the counts of accepted, edited, and rejected diagnoses, the AI-human agreement rate, the number of human corrections, and the number of rule checker failures, with every statistic computed from the actual data files rather than from hard-coded values.
8. To validate the entire system through an automated test suite and a self-verification script that maps each stated requirement to an executable check.

4.5 Scope of the Project

Defining the boundaries of the project was necessary in order to keep it achievable within the available time while still satisfying every requirement of the problem statement.

Within scope. The system covers eight categories of network fault: VLAN configuration, default gateway configuration, DHCP, DNS, routing, access control lists, Network Address Translation, and wireless networking. It operates on evidence supplied as text, comprising a symptom description, a topology note, and show-command output pasted by the user. It produces a structured diagnosis, executes deterministic configuration checks, enforces a human review workflow, records every review decision persistently, and presents summary statistics through a web dashboard. The system is designed for single-user operation on a standard laptop.

Outside scope. The system does not connect directly to live network devices, and it does not execute any configuration command on any device under any circumstances. It only ever recommends commands for a human engineer to run. This is a deliberate safety boundary rather than a limitation of effort: permitting an AI system to apply configuration changes to network infrastructure without human confirmation would introduce precisely the class of risk that the human-in-the-loop design exists to prevent.

Also outside scope are automated packet capture and analysis, real-time network monitoring, support for network operating systems other than Cisco IOS, multi-user concurrent operation with authentication, and any form of automatic remediation. The system additionally does not claim that a fault has been resolved; verification of an applied fix remains a human activity performed on the actual device.

Assumptions. The project assumes that the user has basic familiarity with Cisco IOS command syntax and can interpret show-command output, that the evidence supplied to the system is accurate and has not been altered, and that a human reviewer with appropriate networking knowledge is available to evaluate each diagnosis. The quality of the diagnosis is bounded by the quality and completeness of the evidence supplied.

4.6 Technologies and Tools Used

The technology stack was selected according to a single governing principle: prefer simple, reliable technologies that a student can run on an ordinary laptop over sophisticated infrastructure that adds deployment complexity without adding capability. Consequently the project deliberately avoids containerisation, orchestration platforms, cloud deployment, and database servers.

Table 4.1: Technologies and tools used in the project

Component	Technology	Justification
Programming language	Python 3.11+	Extensive networking and data-handling libraries; standard language for the curriculum; runs natively on Windows, macOS and Linux.
User interface	Streamlit	Produces a professional multi-page web interface from pure Python without any HTML, CSS or JavaScript, allowing effort to be concentrated on diagnostic logic.
Data storage	CSV files	Human-readable, directly inspectable by an evaluator, version-controllable, and requiring no database installation or configuration.
Data manipulation	pandas	Efficient filtering, searching and aggregation of the case dataset and the review logs.
Visualisation	Plotly	Interactive bar and pie charts integrating natively with Streamlit for the dashboard statistics.
AI layer	Anthropic Claude API (optional)	Accessed through a modular interface so the underlying model can be replaced; the system falls back to a labelled demonstration mode when no key is present.
Configuration	python-dotenv	Loads secrets from a .env file that is excluded from version control, ensuring no credential is ever written into source code.
Testing	pytest	Standard Python testing framework used for the twenty-eight automated tests validating the dataset, the rule checker, and the safety guarantees.
Network simulation	Cisco Packet Tracer	Source environment for the laboratory scenarios and the authentic show-command output used throughout the dataset.
Version control	Git	Tracks development history; the .gitignore file ensures the .env credential file is never committed.

The AI layer merits particular explanation. Rather than binding the application to a single model provider, the diagnosis engine exposes a single function that accepts evidence and returns a validated diagnosis object. Within that function, the system first attempts a live model call using an API key read from an environment variable. If no

key is configured, if the required library is unavailable, or if the call fails for any reason including network failure, authentication error, rate limiting, or malformed output, the engine transparently falls back to the demonstration engine rather than raising an error. This design ensures that the application never fails in front of an evaluator because of an external dependency.

4.7 System Architecture

NetSage AI is organised as a layered architecture in which each layer has a single clearly defined responsibility and communicates with adjacent layers through simple, well-defined data structures. The complete architecture is shown in Figure 4.1.

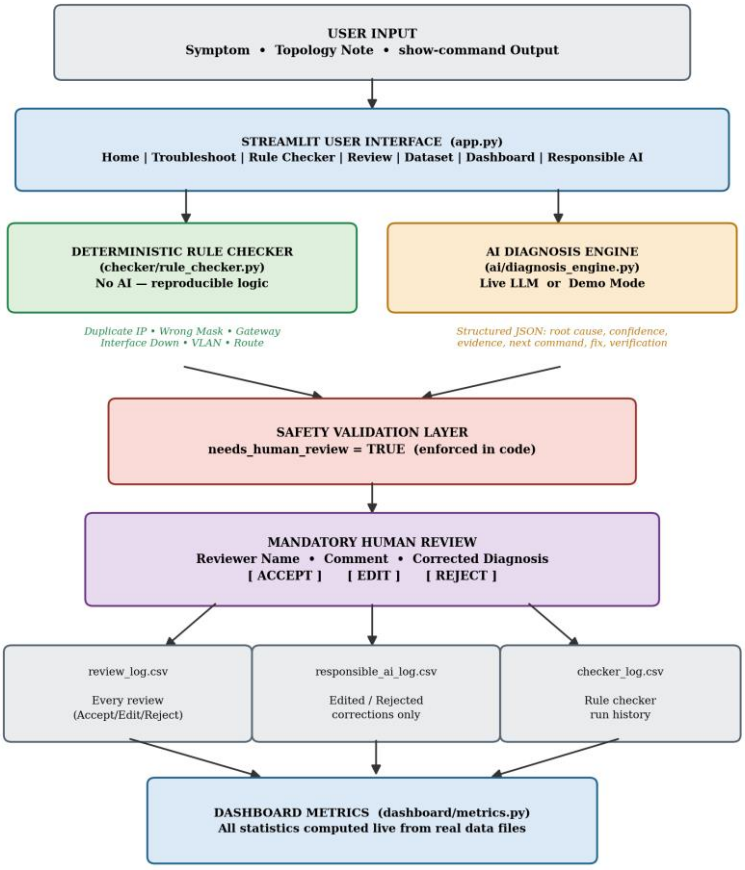


Figure 4.1: System architecture of NetSage AI

The presentation layer is implemented in app.py using Streamlit and provides eight pages: Home, Troubleshoot Case, Rule Checker, Human Review, Case Dataset, Dashboard, Responsible AI, and About Project. This layer is responsible solely for capturing input and rendering output; it contains no diagnostic logic of its own.

The deterministic validation layer, implemented in checker/rule_checker.py, contains no AI component. It parses a simple inventory format describing devices, routes and VLANs, and applies six categories of check. Because this layer is purely algorithmic, its results are perfectly reproducible and can be explained line by line, which makes it a valuable complement to the probabilistic AI layer. Its findings are optionally passed to the AI engine as additional evidence.

The AI diagnosis layer, implemented across ai/diagnosis_engine.py and ai/demo_engine.py, orchestrates the production of a structured diagnosis. It loads the system prompt from the prompt library at runtime, so the AI's behaviour can be modified by editing a markdown file without changing any program code.

The safety validation layer is architecturally the most important component of the system. Every diagnosis, regardless of whether it originated from a live model or from demonstration mode, passes through a normalisation function that validates the presence of every required field, coerces types, constrains the confidence value to the range zero to one, and unconditionally sets the needs_human_review flag to true. The flag is deliberately overwritten irrespective of what the model returned. This means that even a malformed, adversarial, or manipulated model response cannot cause the system to bypass human review, because the guarantee is enforced by program logic rather than by instructions in a prompt that a model may disregard.

The human review layer presents the diagnosis together with a review form requiring a reviewer name and an explicit decision. Where the reviewer selects edit or reject, a corrected diagnosis must be supplied before the review can be submitted.

The persistence layer writes three separate comma-separated value files. The file review_log.csv records every review decision. The file responsible_ai_log.csv records only those reviews in which the AI required correction, forming the Responsible AI record. The file checker_log.csv records the outcome of each rule checker run. Separating these concerns keeps each log independently auditable.

Finally, the metrics layer in dashboard/metrics.py aggregates these files on each page load. No statistic is cached or hard-coded; when a log is empty the corresponding metric correctly reports zero or not applicable rather than displaying a placeholder value.

4.8 Methodology

The project was executed in seven sequential phases, each producing a concrete deliverable that was validated before the next phase began.

Phase one: dataset construction. Thirty troubleshooting cases were authored, each modelled on a scenario reproducible in Cisco Packet Tracer. Care was taken to ensure that every case contained sufficient evidence for the expected diagnosis to be genuinely defensible, and that no case relied on invented command syntax. The dataset is generated by a Python script so that it remains reviewable and extensible as structured data rather than as an opaque spreadsheet. The resulting distribution is shown in Table 4.2.

Table 4.2: Distribution of the troubleshooting case dataset

Issue Type	Number of Cases	Case Identifiers	Representative OSI Layer
VLAN	4	C001 - C004	Layer 2
Default Gateway	4	C005 - C008	Layer 3
DHCP	4	C009 - C012	Application / Layer 3
DNS	3	C013 - C015	Application
Routing	5	C016 - C020	Layer 3
Access Control Lists	4	C021 - C024	Layer 3 / Layer 4
Network Address Translation	3	C025 - C027	Layer 3 / Layer 4
Wireless	3	C028 - C030	Layer 1 / Layer 2
Total	30	C001 - C030	All layers represented

Phase two: prompt engineering. A production system prompt was authored instructing the model to ground every claim in supplied evidence, to never fabricate command output, to separate observation from inference, to lower confidence when evidence is incomplete, to recommend the most useful next diagnostic command, to keep proposed fixes minimal and safe, to never claim that a fix has been applied, and to always mark the diagnosis as requiring human review. An input schema, an output schema, explicit safety constraints, and three complete worked examples were documented alongside it.

Phase three: rule checker development. Six deterministic checks were implemented. During development, the subnet mask check was initially implemented by comparing each device's mask against the most frequently occurring mask in the input. Testing

revealed that this heuristic produced a false positive on a legitimate design in which a point-to-point wide area network link using a /30 mask coexisted with local area networks using /24 masks. The check was therefore redesigned around four principled sub-checks, described in Table 4.3, and regression tests were added to ensure the false positive cannot reappear.

Table 4.3: Deterministic checks implemented in the rule checker

Check	Fault Detected	Method
Duplicate IP	Same address assigned to two devices	Maps every address to the devices holding it and flags any address with more than one holder.
Wrong subnet mask	Invalid, too narrow, or too wide mask	Four sub-checks: structural validity of the mask; host-to-gateway subnet consistency; inclusion of same-VLAN peers; agreement of masks within a single VLAN.
Gateway mismatch	Host pointing at an incorrect gateway	Compares gateway values within each VLAN and warns when a gateway matches no interface recorded as operational.
Interface down	Administratively or operationally disabled port	Inspects reported interface status and assigns higher severity to administratively disabled interfaces.
Missing VLAN	VLAN referenced but never created	Cross-references VLANs referenced by devices against the VLAN registry and its status field.
Missing route	Route entry with no valid next hop	Flags any route whose next hop or outgoing interface is absent or unset.

Phase four: AI engine implementation. The diagnosis engine was implemented with the modular provider interface and the mandatory validation layer described in Section 4.7. The structured output schema enforced by this layer is shown in Table 4.4.

Table 4.4: Structured output schema of the AI diagnosis engine

Field	Type	Purpose
case_id	String	Identifier linking the diagnosis to a dataset case or to custom input.
root_cause	String	The single most likely underlying fault, expressed in one sentence.
confidence	Float (0.0 - 1.0)	Calibrated numeric confidence reflecting the strength of the supporting evidence.
confidence_label	String	Low, Medium or High, derived from the numeric confidence.
osi_layer	String	The OSI layer at which the fault is located.
issue_type	String	The fault category, drawn from the eight supported categories.
evidence	List of strings	Specific lines of supplied show-command output supporting the conclusion.
reasoning_summary	String	Explanation of how the evidence leads to the stated root cause.
next_command	String	The single most useful command to run next to confirm or exclude the hypothesis.
fix_steps	List of strings	Minimal, safe remediation steps for a human engineer to perform.
verification_steps	List of strings	Steps to confirm the fault is resolved after a human applies the fix.
needs_human_review	Boolean	Always true; overwritten in code irrespective of the model's output.

Phase five: human review workflow. The review interface was implemented with mandatory field validation, a guard preventing the same diagnosis from being submitted twice, and automatic propagation of edited and rejected reviews into the Responsible AI log.

Phase six: Responsible AI documentation. Six cases were documented in which the AI diagnosis required correction. These include an instance in which the model prioritised a DNS hypothesis despite access control list output containing an explicit deny rule matching the affected subnet, and an instance in which the model assigned high confidence to a router misconfiguration hypothesis that the supplied interface output directly contradicted.

Phase seven: testing and verification. Twenty-eight automated tests were written covering dataset integrity, schema conformance, the human review safety guarantee, demonstration mode labelling and coverage, log persistence, and all six rule checker categories. A separate self-verification script was written that maps each requirement of the problem statement to an executable check and reports a pass or fail result for each.

4.9 Expected Outcomes

The project produced a complete, runnable application together with its supporting dataset, documentation, and test suite. The outcomes are described below and the verification results are summarised in Table 4.5.

A validated case dataset. Thirty troubleshooting cases were produced covering all eight required fault categories with a balanced distribution, each containing complete symptom, topology, evidence, expected fault, OSI layer, concept tag and verification fields.

A functioning diagnosis pipeline. Every case in the dataset produces a complete, well-formed structured diagnosis. An earlier iteration of the demonstration engine covered only seven cases and degraded to a weak keyword-matching response for the remaining twenty-three; this was identified during review and corrected so that all thirty cases now yield a full diagnosis, clearly labelled as to its origin and capped at medium confidence so that a dataset lookup is never mistaken for model reasoning.

A verified deterministic checker. All six checks operate correctly, and the bundled sample input deliberately seeds one fault of every category so that a single execution demonstrates the complete range of the checker's capability.

An unbypassable oversight mechanism. The requirement for human review is enforced in program code. Automated testing confirms that the review flag is set to true across every diagnosis path, including deliberately malformed inputs.

A transparent record of AI error. Six documented correction cases exceed the required minimum of five, and the log grows automatically as further reviews are recorded.

Table 4.5: Summary of project verification results

Verification Criterion	Target	Achieved	Status
Troubleshooting cases in dataset	30 minimum	30	Met
Fault categories covered	8	8	Met
Cases producing a complete diagnosis	All	30 of 30	Met
Deterministic checks implemented	6	6	Met
Check categories demonstrated by sample input	6	6	Met

Verification Criterion	Target	Achieved	Status
Responsible AI correction cases logged	5 minimum	6	Exceeded
Automated unit tests passing	All	28 of 28	Met
Requirement verification checks passing	All	22 of 22	Met
Hard-coded credentials in source code	0	0	Met
Diagnoses accepted without human review	0	0	Met

Beyond these measurable outcomes, the project yielded a methodological result worth recording. The most instructive defects encountered during development were not program crashes but silent correctness failures: a subnet mask check that produced a confident and entirely wrong result on a legitimate network design, and a demonstration mode that appeared to work while quietly degrading for the majority of cases. Both were found only by deliberately attempting to break the system rather than by confirming that it worked on the inputs it was designed for. This mirrors the project's own central argument: an automated system that appears fluent and confident requires independent verification precisely because its failures are not self-announcing.

4.10 Certificates of Completion and Communication Proof

[Attach in this section the scanned copy of the internship completion certificate, together with any supporting communication or email correspondence relating to the internship, as required by the prescribed report format.]

Suggested items to attach:

- Internship offer or acceptance letter.
- Internship completion certificate issued by the organisation.
- Email correspondence confirming the project allocation and the problem statement.
- Any interim feedback, evaluation, or acknowledgement received during the internship period.

CHAPTER 5: BIBLIOGRAPHY / REFERENCES

[1] Cisco Systems, Inc. (2024). Cisco Packet Tracer: Networking Simulation Tool. Cisco Networking Academy. Retrieved from https://www.netacad.com/courses/packet-tracer

[2] Cisco Systems, Inc. (2023). CCNA 200-301 Official Cert Guide, Volume 1 and Volume 2. Cisco Press, Indianapolis, Indiana.

[3] Cisco Systems, Inc. (2024). Configuring VLANs and VLAN Trunking Protocol. Cisco IOS Configuration Guides. Retrieved from https://www.cisco.com/c/en/us/support/docs/lan-switching/vlan/

[4] Cisco Systems, Inc. (2024). Configuring Network Address Translation: Getting Started. Cisco Technical Documentation. Retrieved from https://www.cisco.com/c/en/us/support/docs/ip/network-address-translation-nat/

[5] Cisco Systems, Inc. (2024). Configuring IP Access Lists. Cisco Technical Documentation. Retrieved from https://www.cisco.com/c/en/us/support/docs/security/ios-firewall/

[6] Odom, W. (2020). Computer Networking: A Top-Down Approach to CCNA Concepts. Pearson Education, London.

[7] Kurose, J. F. and Ross, K. W. (2021). Computer Networking: A Top-Down Approach, 8th Edition. Pearson Education, Boston.

[8] Tanenbaum, A. S. and Wetherall, D. J. (2021). Computer Networks, 6th Edition. Pearson Education, Boston.

[9] Streamlit Inc. (2024). Streamlit Documentation: Building Data Applications in Python. Retrieved from https://docs.streamlit.io

[10] Anthropic PBC. (2024). Claude API Documentation: Messages API and Prompt Engineering Guide. Retrieved from https://docs.anthropic.com

[11] Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library Documentation. Retrieved from https://plotly.com/python/

[12] The pandas development team. (2024). pandas: Powerful Python Data Analysis Toolkit. Retrieved from https://pandas.pydata.org/docs/

[13] Krupp, A. and pytest-dev team. (2024). pytest Documentation: Testing Framework for Python. Retrieved from https://docs.pytest.org

[14] Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., et al. (2019). Guidelines for Human-AI Interaction. Proceedings of the CHI Conference on Human Factors in Computing Systems, pages 1-13.

[15] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, Volume 55, Issue 12, pages 1-38.

[16] Bansal, G., Wu, T., Zhou, J., Fok, R., Nushi, B., et al. (2021). Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. Proceedings of the CHI Conference on Human Factors in Computing Systems.

[17] National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST, U.S. Department of Commerce.

[18] Postel, J. (1981). Internet Protocol, DARPA Internet Program Protocol Specification. RFC 791, Internet Engineering Task Force.

[19] Droms, R. (1997). Dynamic Host Configuration Protocol. RFC 2131, Internet Engineering Task Force.

[20] Mockapetris, P. (1987). Domain Names - Concepts and Facilities. RFC 1034, Internet Engineering Task Force.